



I

**Assignment: Project Proposal: Feasibility Report and Preliminary Requirement Analysis**

**Project Title:** FistBump Living Resume and Job Targeter

**Authors:** Tyler McGuire, Tyler Goodman, Joe Cooney, Aron Magnusson

**Course:** CSCI 362 - Software Engineering

**Date:** September 8, 2026

## **Table of Contents**

1. Project Idea
2. Market Survey
3. Stakeholders
4. Functional Requirements
5. Non-Functional Requirements
6. Proposed System Design
7. Process to Follow
8. Project Timeline and Deliverables
9. Risk Analysis
10. Development Tools
11. Tools and Skills to be Developed
12. Budget
13. Backup Plan
14. Conclusion
15. Appendix A

## **1. Project Idea**

In July 2026, the U.S. unemployment rate remained at 4.0%, representing approximately 6.9 million job-seeking individuals. To secure a single job offer, candidates frequently submit between 21 and 80 applications spread across fragmented platforms such as Indeed, LinkedIn, Handshake, USAJOBS, and ClearanceJobs. Managing deadline dates, customized resume versions, and job

requirements across dozens of applications creates significant operational overhead and cognitive fatigue.

The **FistBump Living Resume and Job Targeter** is a local-first, cross-platform Electron application running natively on Windows, macOS, and Linux operating systems. The application parses job postings locally to identify required qualifications, performs side-by-side skill-gap comparisons against a master resume, and executes controlled local artificial intelligence (AI) model inference to generate reviewable bullet-point revisions. Users retain full authority to accept, modify, or reject AI suggestions before exporting tailored resumes as Portable Document Format (PDF) files, keeping their sensitive personal data stored securely on their local host device.

## 2. Market Survey

| Platform                                  | Summary                                                                                        | Pros                                                                          | Cons                                                                                                                      |
|-------------------------------------------|------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------|
| <a href="#"><u>Teal HQ</u></a>            | Cloud web dashboard and browser extension to save jobs, build resumes, and track applications. | Clear interface; tracks contacts and application stages; easy browser saving. | Keyword analysis and tailoring require paid subscriptions; relies heavily on cloud synchronization.                       |
| <a href="#"><u>Resume Tailor Tool</u></a> | Web utility scanning job descriptions against uploaded resumes to highlight missing keywords.  | Fast keyword gap identification; simple match score output.                   | No application pipeline tracking; forces external job management; processes data on remote servers.                       |
| <a href="#"><u>JobTailor</u></a>          | Web writing assistant generating customized application materials using cloud AI.              | Covers both resumes and cover letters.                                        | Low human-in-the-loop control; risks unreviewed inaccurate text; no desktop application; recurring subscription required. |
| <a href="#"><u>Huntr</u></a>              | Job search organization platform with spreadsheet boards, bookmarking extensions, and metrics. | Strong visual stage tracking; browser extension support; detailed metrics.    | Most features require a paid subscription; restricted AI tailoring on free plans; cloud dependency.                       |
| <a href="#"><u>ApplyBoost AI</u></a>      | Web service designed to reword resume bullet points and adjust keyword alignment.              | Quick bullet point rewrites for target job requirements; fast match scoring.  | Recurring subscription required; limited formatting choices; lacks pipeline management tools.                             |

|                                          |                                                                                                       |                                                                                  |                                                                                                      |
|------------------------------------------|-------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------|
| <a href="#"><u>GetJobTailor</u></a>      | Web tool where users paste job text and resume sections side-by-side for comparison.                  | Simple setup without browser extensions; quick side-by-side feedback.            | No application tracking; manual copy-pasting required for updates; no local storage.                 |
| <a href="#"><u>Resume AI</u></a>         | Automated platform handling job matching, resume rewrites, and application tasks via cloud AI.        | Automates much of the application process; includes interview prep modules.      | Lacks human-in-the-loop oversight; risks incorrect AI output without review; expensive subscription. |
| <a href="#"><u>Free Resume Tools</u></a> | Free web utility providing keyword matching and basic formatting checks without account registration. | Free to use; no account required; fast feedback.                                 | Cannot save state; lacks application tracking; minimal automated assistance.                         |
| <a href="#"><u>Simplify</u></a>          | Browser extension and platform autofilling application forms, parsing postings, and tracking jobs.    | Useful auto-logging of applications and autofilling forms on popular job boards. | Recurring subscription required; cloud dependency.                                                   |
| <a href="#"><u>Jobscan</u></a>           | Job application scanner scoring resume compatibility against postings and checking formatting.        | Good keyword match scoring; detailed formatting and parsing.                     | Recurring subscription required; strict limits on free users; no application tracking board.         |
| <a href="#"><u>Jobright</u></a>          | AI job discovery tool matching candidates to listings and assisting with keyword tailoring.           | Strong matching search; contact parsing.                                         | Recurring subscription required; low human-in-the-loop oversight; cloud dependency.                  |

### Existing Job Search Tools Overview:

- **Pros:** Visual stage tracking, form autofilling via browser extensions, and rapid response times.
- **Cons:** Recurring monthly subscriptions, third-party cloud data privacy risks, low levels of human-in-the-loop integration, risk of inaccurate AI inference without human review, and fragmented feature sets requiring users to juggle multiple tools.

### How FistBump Addresses Market Limitations:

- **Data Privacy:** Runs locally via Electron, keeping all resumes, application records, and AI model inference strictly on the user's host device or self-sourced compute infrastructure.
- **No Recurring Subscriptions:** Built with zero inherent cloud dependency, eliminating the need for paid subscription tiers.
- **Controlled AI Assistance:** Enforces a mandatory human-in-the-loop review, requiring users to explicitly accept, reject, or modify AI-generated edits before export.
- **Unified Feature Set:** Combines posting parsing, skill-gap analysis, controlled AI revisions, PDF exports, and application pipeline tracking into a single native desktop application.

### 3. Stakeholders

- **Job Seekers (Primary End-Users):** Candidates who require an organized, privacy-preserving desktop platform to track job applications, analyze postings, and efficiently generate tailored resumes.
- **Recruiters and Business Owners (Secondary Stakeholders):** Hiring teams who benefit indirectly by receiving accurately tailored, highly relevant, and well-structured resumes that directly match candidate qualifications to job requirements.
- **Developers (Engineering Team):** Project authors responsible for designing system architecture, implementing Inter-Process Communication (IPC) bridges, managing backend services, configuring local Large Language Model (LLM) bindings, and delivering an operational product.
- **Academic Evaluation Team (Course Instructor and Classmates):** Evaluators assessing software engineering methodologies, design robustness, code quality, and delivery milestones against course rubrics.

### 4. Functional Requirements

- **FR-1:** The system shall allow users to store, edit, and maintain a structured master resume on their local device or self-sourced compute infrastructure.
  - Who: The system
  - Shall: Allow users to store, edit, and maintain a structured master resume on their local device or self-sourced compute infrastructure.
  - By: Maintaining local databases and processing options.
  - In order to: Provide certainty of where user data is located/processed.
- **FR-2:** The system shall parse job posting text to identify required qualifications and compare them against the user's master resume to highlight skill gaps.
  - Who: The system
  - Shall: Parse job posting text
  - By: Using a recursive algorithm to extract text
  - In order to: identify required qualifications and compare them against the user's master resume
- **FR-3:** The system shall provide locally processed AI-assisted revisions tailored to specific positions, allowing users to review, reject, or modify changes.
  - Who: The system
  - Shall: Provide locally processed revisions tailored to specific positions, allowing users to review, reject, or modify changes
  - By: Implementing local AI inference.
  - In order to: Provide private, user-verified revisions.
- **FR-4:** The system shall compile accepted resume revisions into PDF documents and export them to the local file system.
  - Who: The system

- Shall: Compile resume revisions into PDF documents for export.
  - By: Using the Go PDF compilation library.
  - In order to: Provide output to users.
- **FR-5:** The system shall store all job entries, application statuses, and resume versions locally without initiating external network calls.
  - Who: The system
  - Shall: Store all job entries, application statuses, and resume versions locally.
  - By: Maintaining a local database.
  - In order to: Keep user data private.
- **FR-6:** The system should be compatible with Linux, Windows, and macOS.
  - Who: The system
  - Should: be compatible with Linux, Windows, and macOS
  - By: Being built through the Electron framework
  - In order to: Reach more users.

## 5. Non-Functional Requirements

- **NFR-1 (Security and Data Privacy):** All resume data, job postings, and AI model inference must remain entirely on the user's local device or within their own supplied compute infrastructure.
  - Who: Resume and job data, AI model inference.
  - Shall: Remain entirely on the user's local device or within their own supplied compute infrastructure.
  - By: Using an on-device model or allowing users to supply their own key and storing information in local databases.
  - In order to: Protect user data and privacy.
- **NFR-2 (Cross-Platform Portability):** The application must build and operate natively across Windows, Linux, and macOS environments.
  - Who: The application.
  - Shall: Have the ability to function across a multitude of OS.
  - By: Using electron which can compile in different file types.
  - In order to: Allow for cross platform compatibility.
- **NFR-3 (Performance and Latency):** Skill-gap analysis and keyword extraction must complete with minimal execution latency (under 1.5 seconds).
  - Who: Skill-gap analysis and keyword extraction.
  - Shall: Complete with minimal execution latency.
  - By: Using optimized parsing algorithms, keyword indexing, parallel processing
  - In order to: Maintain a responsive and real time experience to prevent interruption
- **NFR-4 (Usability and Interactivity):** The user interface must provide clear side-by-side text difference (diff) views to ensure users retain total control over final resume text.
  - Who: The User.
  - Shall: Have the ability to view changes made to their resume.
  - By: Highlighting changes made to the master resume on the final resume.
  - In order to: Show the user what changes our application made to the resume for review.
- **NFR-5 (Resource Efficiency):** The application package must maintain efficient local storage utilization for application records and database files (under 250 MB installation footprint excluding local model weights).
  - Who: The application package
  - Shall: Maintain efficient local storage utilization for application records and database files

- By: Efficient design and garbage collection with user controls
- In order to: Minimize storage costs and prevent disk space exhaustion

## 6. Proposed System Design

FistBump utilizes an Electron desktop wrapper backed by a compiled, high-performance Go background process to handle data persistence, string parsing, and AI orchestration securely.

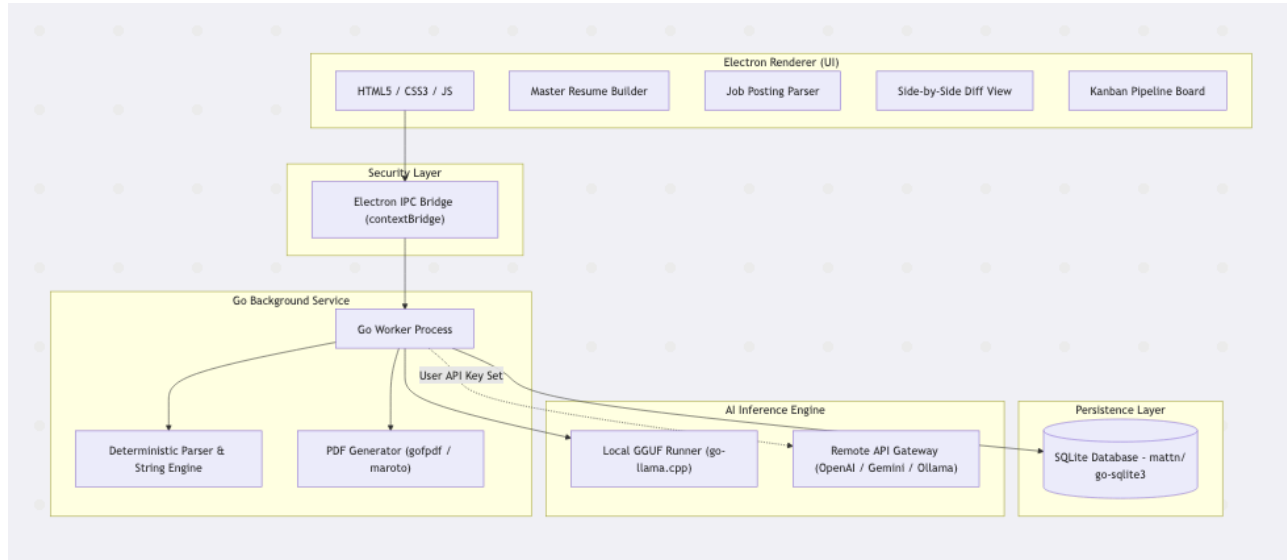


Figure 1: High-level architectural flowchart showing the relationship between the Electron Frontend, IPC Security Bridge, Go Backend, SQLite Storage, and AI Inference Engine.

- **Frontend:** HTML5, CSS3, and JavaScript (ES6+) driving a dual-panel Graphical User Interface (GUI) for resume management, side-by-side diff review, and spreadsheet stage tracking.
- **IPC Bridge:** Electron Inter-Process Communication (IPC) bridge using context isolation to execute secure, isolated asynchronous calls between the user interface renderer and backend services.
- **Backend:** Embedded Go execution runtime managing local Structured Query Language (SQLite) relational storage, deterministic string parsing, side-by-side diffing logic, and native PDF document generation.
- **AI Provider:**
  - *Local Inference:* Embedded bindings executing quantized model weights directly on host hardware for complete offline processing.
  - *Remote API Gateway:* Secure Application Programming Interface (API) client dispatching prompts to external LLM providers when user-configured API keys are explicitly supplied.

## 7. Process to Follow (Why?)

**Agile Two-Iteration Methodology:** An Agile framework systematically reduces architectural risk, validates local hardware AI execution early, and ensures consistent milestone progress toward course rubrics.

- **1st Iteration (Core Foundation and Feasibility Verification):**
  - *Focus:* Local database schema configuration, master resume Create, Read, Update, and Delete (CRUD) interface, job posting parsing, deterministic keyword extraction,

- and preliminary local AI inference integration via Go bindings.
- *Why*: Validates the local-first storage architecture and verifies that embedded hardware inference functions smoothly across target platforms without locking the main user interface thread.
- **2nd Iteration (Feature Integration, Refinement, and Delivery):**
  - *Focus*: Side-by-side diff component, PDF compilation engine, application tracking pipeline (spreadsheet board), fallback API gateway integration, and system testing.
  - *Why*: Integrates human-in-the-loop controls into a polished interface, ensuring all functional requirements and local data privacy constraints are fully realized before final presentation.

## 8. Project Timeline and Deliverables

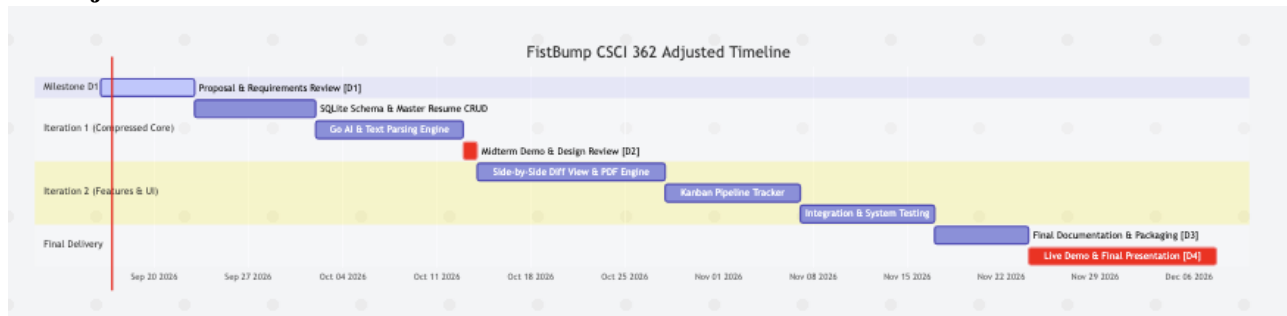


Figure 2: Project implementation schedule illustrating sprint timelines, iteration milestones, and final delivery deadlines.

| Phase / Activity                                    | Timeline  | Deliverables and Milestones                                                                                      |
|-----------------------------------------------------|-----------|------------------------------------------------------------------------------------------------------------------|
| <b>Requirements and Architecture Setup</b>          | Weeks 1–2 | Initial system architecture, environment setup, and repository initialization.                                   |
| <b>Proposal and Feasibility Report</b>              | Week 2    | <b>Deliverable 1 (D1 - Sep 8):</b> Project Proposal, Requirement Analysis, and Feasibility Report submission.    |
| <b>Iteration 1: Database and Master Resume Core</b> | Weeks 3–4 | Database schema implementation and master resume CRUD interface.                                                 |
| <b>Iteration 1: Go AI and Parsing IPC</b>           | Weeks 4–5 | Go text parsing algorithms, IPC bridge routing, and local AI binding tests.                                      |
| <b>Midterm Demo and Design Review</b>               | Week 5    | <b>Deliverable 2 (D2 - Oct 6):</b> Working Midterm Prototype (Resume CRUD, parsing, basic database persistence). |
| <b>Iteration 2: Diff View and PDF Engine</b>        | Weeks 6–7 | Interactive side-by-side diff user interface component and Go PDF compilation engine.                            |
| <b>Iteration 2: Pipeline Tracker</b>                | Weeks 7–8 | spreadsheet application tracking pipeline and stage management features.                                         |
| <b>Integration and Performance Testing</b>          | Weeks 8–9 | Cross-platform build verification, hardware latency testing, and fallback checks.                                |

|                                          |             |                                                                                                                |
|------------------------------------------|-------------|----------------------------------------------------------------------------------------------------------------|
| <b>Final Documentation and Packaging</b> | Week 9      | <b>Deliverable 3 (D3 - Nov 10):</b> Release binaries (.exe / .dmg / .AppImage) and final user documentation.   |
| <b>Live Demo and Final Presentation</b>  | Weeks 11–12 | <b>Deliverable 4 (D4 - Nov 24):</b> Final Presentation, Live System Demonstration, and Team Rubric Evaluation. |

## 9. Analysis

Evaluating potential failure points ensures the project stays on schedule and meets software quality standards. Risks are categorized into schedule, resource, and functionality domains:

- Schedule Risk:** Given that this application must be fully operational by the end of the academic term, technical delays pose a major concern. Integrating local AI inference wrappers or configuring cross-platform builds can consume significant development cycles. Scope creep midway through the project could also force architecture redesigns.
  - Mitigation:* The team will strictly follow a bi-weekly sprint model, prioritizing database operations, basic parsing, and master resume revisions before polishing advanced AI features.
- Resource Risk:** Because FistBump runs a compiled binary across diverse user operating systems, hardware specifications represent a risk. Development machines vary in processing power and memory capacity, which can lead to compilation errors or execution latency when running quantized models. Reliance on remote Git repositories may introduce brief delays during network outages. Additionally, team members must balance varying academic commitments across the semester.
  - Mitigation:* The team will utilize Discord, Trello, and GitHub to coordinate tasks, execute early multi-platform build checks, and maintain clear code documentation.
- System Functionality Risk:** Running compute- and memory-intensive tasks—such as AI inference, natural language processing, or PDF generation—could introduce latency or interface stuttering, degrading user experience.
  - Mitigation:* FistBump will prevent interface locking by running heavy operations inside isolated Go worker processes. Clear system requirements will be communicated to users. If a user's host machine encounters severe resource contention, FistBump will automatically detect the bottleneck and fall back to lightweight, deterministic algorithms—guaranteeing fast, operational execution across all hardware tiers.

## 10. Development Tools

- Languages and Technologies:** Go, JavaScript (ES6+), HTML5, CSS3, Structured Query Language (SQL).
- Application Framework:** Electron.js desktop application runtime.
- Frontend:** Vanilla JavaScript, component-based user interface architecture, CSS Flexbox and Grid, custom side-by-side diff viewer.
- Backend:** Compiled Go executable binary, SQLite embedded database, Go native PDF compilation engine, local C/C++ model execution bindings.
- Collaboration and Workflow:** Discord (team communication), Trello (Agile task management and kanban tracking), Git and GitHub (version control, feature branching, and pull requests).

## 11. Tools and Skills to be Developed

- Building cross-platform desktop applications using Electron's secure multi-process architecture.
- Integrating native C/C++ execution bindings in Go for offline LLM execution.
- Architecting localized relational database schemas and migration workflows with SQLite.
- Programmatically generating formatted PDF documents and calculating text diff representations in Go.

## 12. Budget

As the product operates locally on the user's host machine using open-source utilities and user-provided API credentials, no operational expenses are required.

## 13. Backup Plan

- **Source Code Control:** Source code and documentation will be maintained in a Git version-controlled repository with automated multi-branch backups.
- **Algorithmic Fallback:** If local AI inference fails due to hardware constraints, missing system drivers, or memory limits, or if external API keys are invalid or unreachable, the application defaults immediately to deterministic keyword-matching algorithms. This guarantees that core skill-gap analysis, resume editing, and PDF exporting remain fully functional regardless of hardware state.

## 14. Conclusion

The job application process remains highly fragmented, inefficient, and reliant on recurring cloud subscriptions that expose sensitive candidate data to third-party risks. The **FistBump Living Resume and Job Targeter** resolves these core challenges by combining application pipeline tracking, automated job parsing, skill-gap analysis, and AI-assisted tailoring into a single, cohesive desktop platform. By enforcing a local-first architecture and mandatory human-in-the-loop review, FistBump guarantees total privacy while ensuring users maintain full agency over every exported application. Supported by a deterministic fallback engine and a structured Agile development timeline, the project is uniquely positioned to deliver an operational, reliable, and high-impact solution that elevates the job-seeking experience without compromising personal data.

## Appendix A: Supplemental Materials and Implementation Reference

### Appendix A.1: Data Storage Schema Overview

To ensure complete local execution, all user records are persisted within an embedded SQLite database stored directly on the host file system. The underlying relational schema uses the following entity structure:

- **Master\_Resumes:** Stores raw markdown sections, base contact information, core skill tags, and baseline experience entries.
- **Job\_Postings:** Stores targeted job listing metadata, raw description strings, identified skill requirements, and extraction timestamps.

- **Tailored\_Resumes:** Maps target job postings to customized resume variants, storing user-accepted delta edits and generated output files.
- **Application\_Pipeline:** Tracks application status stages (e.g., *Saved, Applied, Interviewing, Offered, Archived*) along with company names, submission dates, and custom notes.

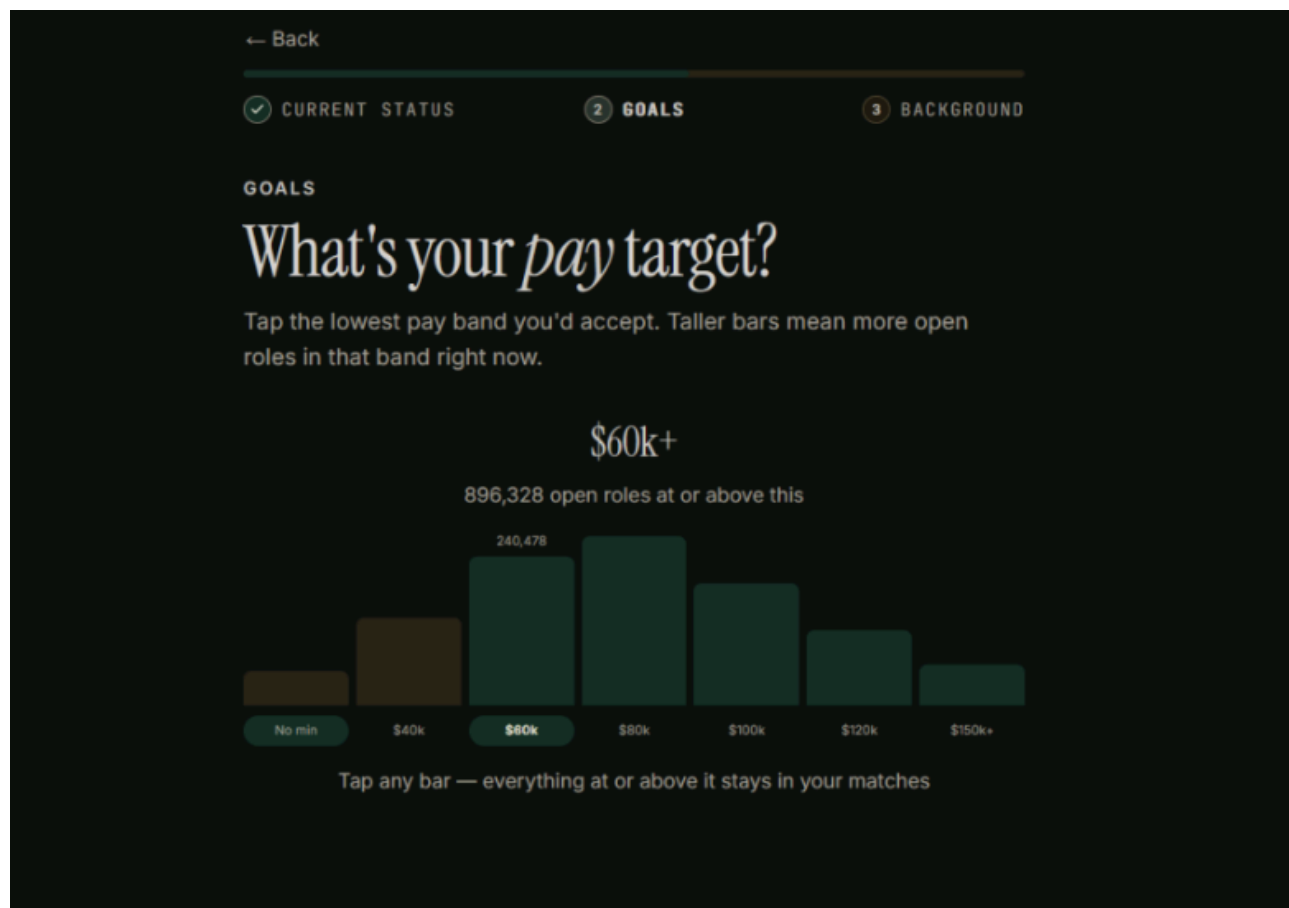
**Appendix A.2: System Hardware Requirements**

- **Supported Operating Systems:** Windows 10/11 (64-bit), macOS 12+ (Intel and Apple Silicon), Linux (Ubuntu 20.04+, Fedora 36+, Arch Linux).
- **Minimum System Specs:** 4 GB RAM, Dual-Core x86/ARM Processor, 250 MB available storage (for native app binary without local LLM models).
- **Recommended Specs (for Local Model Inference):** 16 GB RAM, Quad-Core CPU, Dedicated GPU / Apple Neural Engine, 4 GB available storage (for quantized GGUF model storage).

**Appendix A.3: Algorithmic Fallback Matrix**

The fallback engine ensures core functionality continues without interruption if system resources are constrained or APIs are unavailable:

| Functionality                | Primary Execution Path                                 | Automatic Fallback Path                                     |
|------------------------------|--------------------------------------------------------|-------------------------------------------------------------|
| <b>Skill-Gap Analysis</b>    | Local GGUF LLM inference via <code>go-llama.cpp</code> | Deterministic N-gram string matching and keyword extraction |
| <b>Bullet Point Revision</b> | User-configured External API (OpenAI/Gemini)           | Local quantized model or structured rule-based templates    |
| <b>PDF Generation</b>        | Native Go compilation ( <code>gofpdf/maroto</code> )   | Local plain text / Markdown export                          |



Standout

← Back

✓ CURRENT STATUS 2 GOALS 3 BACKGROUND

### GOALS

# What kind of *work*?

Browse fields on the left — tap roles on the right to add them to your picks.

Search fields or roles, or type your own

YOUR PICKS • Nothing selected yet — tap roles below to add them

- Project Management
- Software Engineering
- ML / AI
- Data
- Cybersecurity
- Product
- Design
- Marketing
- 33 MORE FIELDS

Project Management

Project / program manager, scrum, PMO, coordinator

Select all Project Management

PROJECT & PROGRAM

Project Manager Program Manager Project Coordinator

Delivery Manager Portfolio Manager

TECHNICAL & IT PROJECTS

Technical Program Manager Technical Project Manager

IT Project Manager

AGILE & PRO

Scrum Master PMO Analyst

